

Conjuntos em Go 1.28

Luciano Ramalho

luciano@ramalho.org

github/codeberg: @ramalho



Meu melhor trabalho (até hoje): Fluent Python

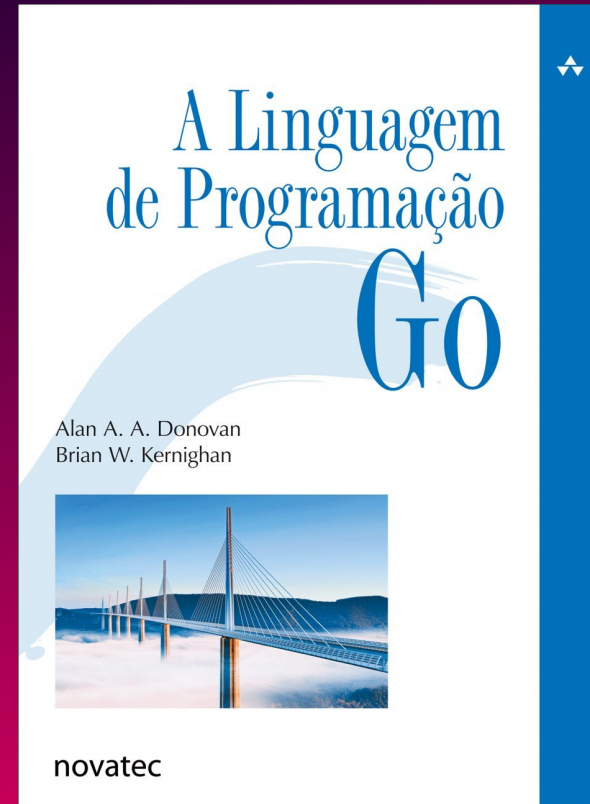


- Publicado em 9 idiomas, 2 edições (2015, 2022)
- Python idiomático, explorando o design da linguagem e sua biblioteca padrão



Luciano Ramalho

- Membro da ação Bartz v. Anthropic
 - E assinante do Claude Code
- Principal consultant na Thoughtworks (2015-2023)
- Revisor técnico da edição brasileira do GOPL (2016)
- Co-fundador do Garoa Hacker Clube (desde 2010)





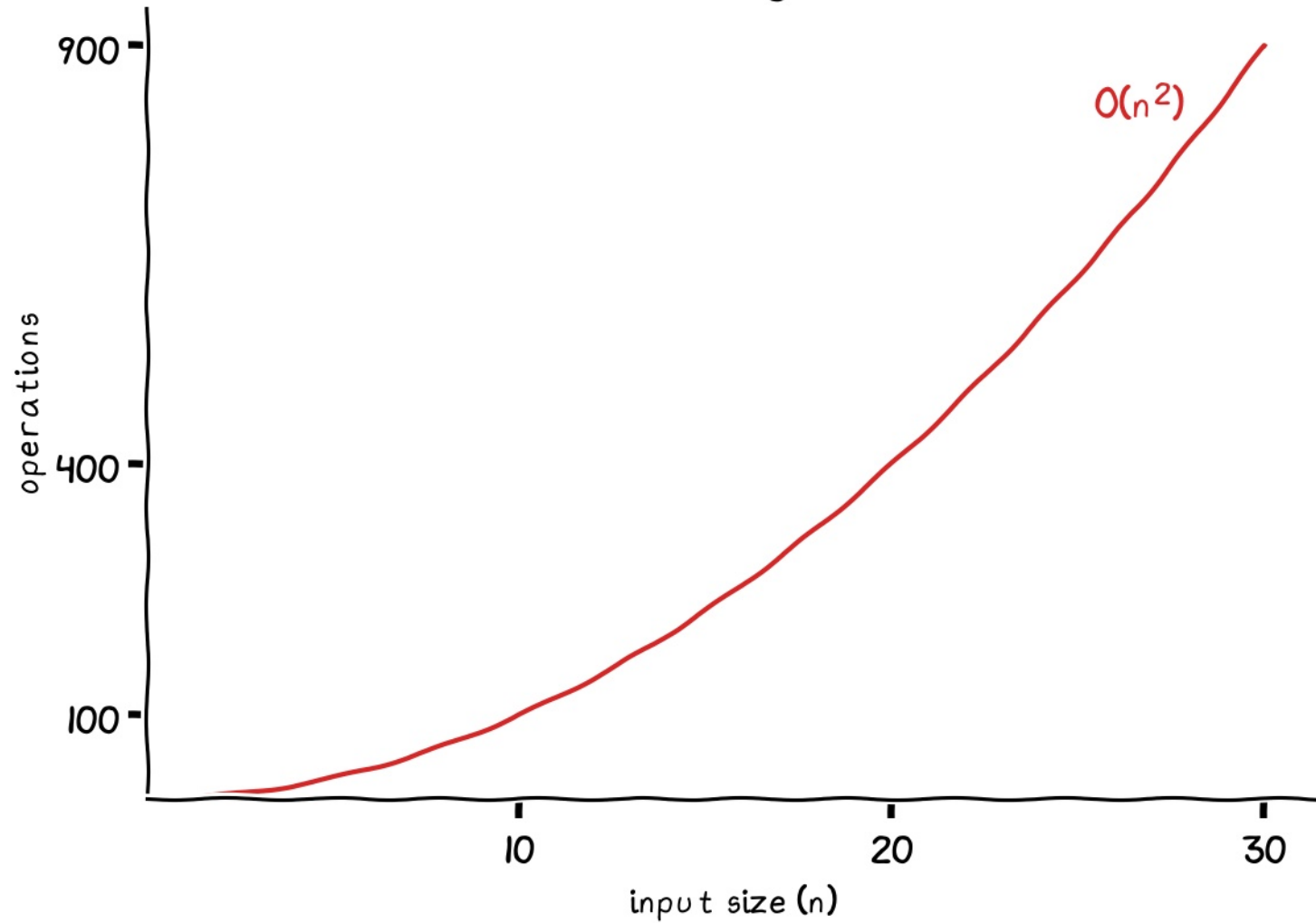
Exemplo: unigo

unigo: um exemplo simples e prático

- Inspirado no comando `uniq`, que elimina linhas duplicadas
- Diferença importante:
 - `uniq` só elimina duplicatas que ocorrem em linhas sucessivas (necessário fazer `sort` antes)
 - `unigo` elimina todas as linhas duplicadas e preserva a ordem relativa
- Usando uma busca linear, `unigo` é inviável: custo $O(n^2)$



Quadratic growth



unigo em $O(n)$

Padrão tradicional: map[string]struct{}

```
1 // writeUnique copies input to output,  
2 // keeping only the first occurrence of each line.  
3 func writeUnique(input io.Reader, output io.Writer) error {  
4     lines := bufio.NewScanner(input)  
5     buf := bufio.NewWriter(output)  
6  
7     seen := make(map[string]struct{})  
8     for lines.Scan() {  
9         line := lines.Text()  
10        if _, dup := seen[line]; !dup { // look it up...  
11            seen[line] = struct{}{} // ...then store it  
12            fmt.Fprintln(buf, line)  
13        }  
14    }  
15  
16    // Report read error if it happens; flush always  
17    return cmp.Or(lines.Err(), buf.Flush())  
18 }
```


Outro padrão tradicional: map[string]bool

```
1 // writeUnique copies input to output,  
2 // keeping only the first occurrence of each line.  
3 func writeUnique(input io.Reader, output io.Writer) error {  
4     lines := bufio.NewScanner(input)  
5     buf := bufio.NewWriter(output)  
6  
7     seen := make(map[string]bool)  
8     for lines.Scan() {  
9         line := lines.Text()  
10        if !seen[line] { // zero value false means "not seen"..  
11            seen[line] = true // ...then store it  
12            fmt.Fprintln(buf, line)  
13        }  
14    }  
15  
16    // Report read error if it happens; flush always  
17    return cmp.Or(lines.Err(), buf.Flush())  
18 }
```

Futuro: set.Set[string]

```
1 // writeUnique copies input to output,  
2 // keeping only the first occurrence of each line.  
3 func writeUnique(input io.Reader, output io.Writer) error {  
4     lines := bufio.NewScanner(input)  
5     buf := bufio.NewWriter(output)  
6  
7     seen := make(set.Set[string])  
8     for lines.Scan() {  
9         line := lines.Text()  
10        if seen.Insert(line) { // true when set changed  
11            fmt.Fprintln(buf, line)  
12        }  
13    }  
14  
15    // Report read error if it happens; flush always  
16    return cmp.Or(lines.Err(), buf.Flush())  
17 }
```

```
// unigo-map
seen := make(map[string]struct{})
for lines.Scan() {
    line := lines.Text()
    if _, dup := seen[line]; !dup {
        seen[line] = struct{}{}
        fmt.Fprintln(buf, line)
    }
}
```

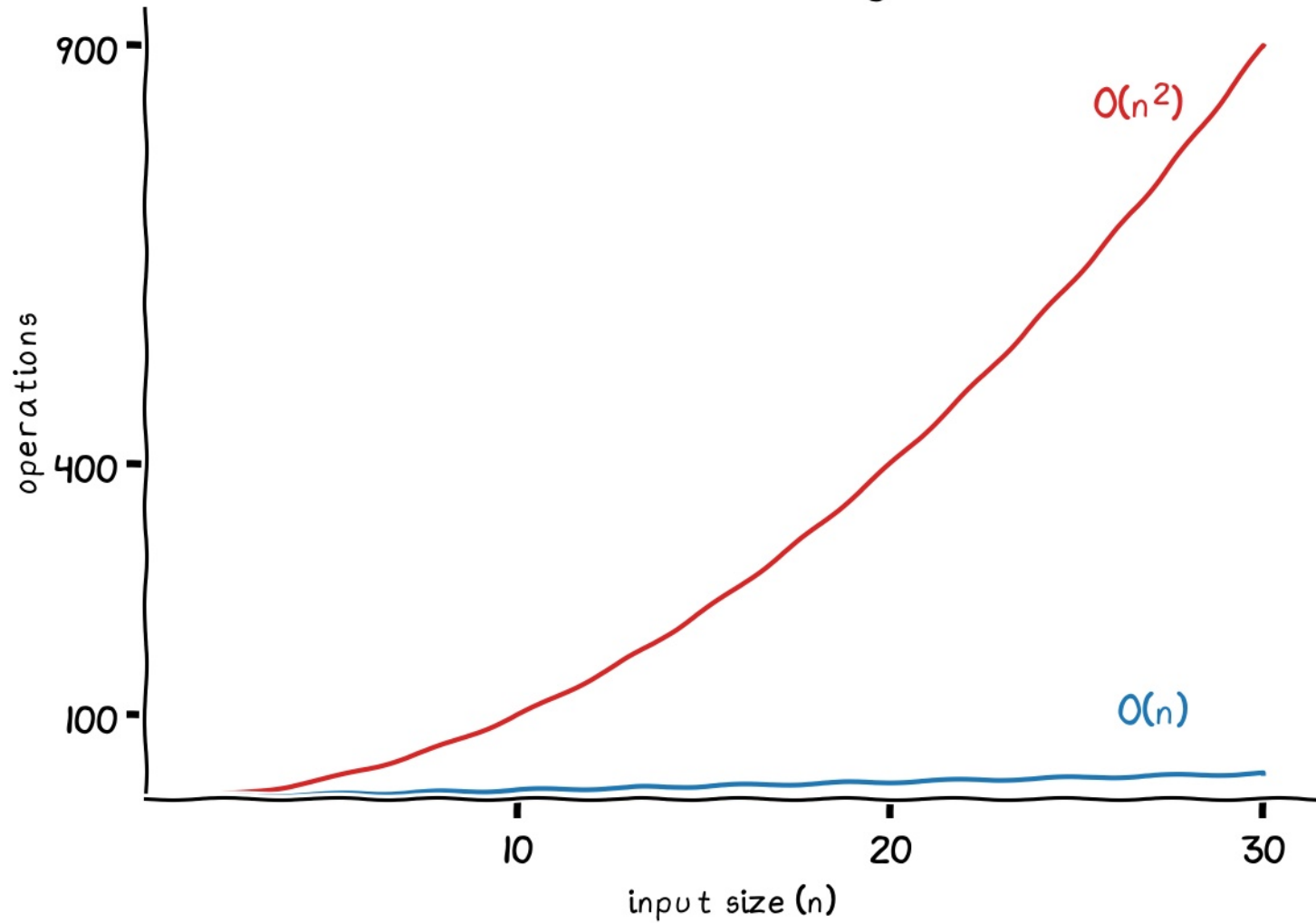
```
// unigo-mapbool
seen := make(map[string]bool)
for lines.Scan() {
    line := lines.Text()
    if !seen[line] {
        seen[line] = true
        fmt.Fprintln(buf, line)
    }
}
```

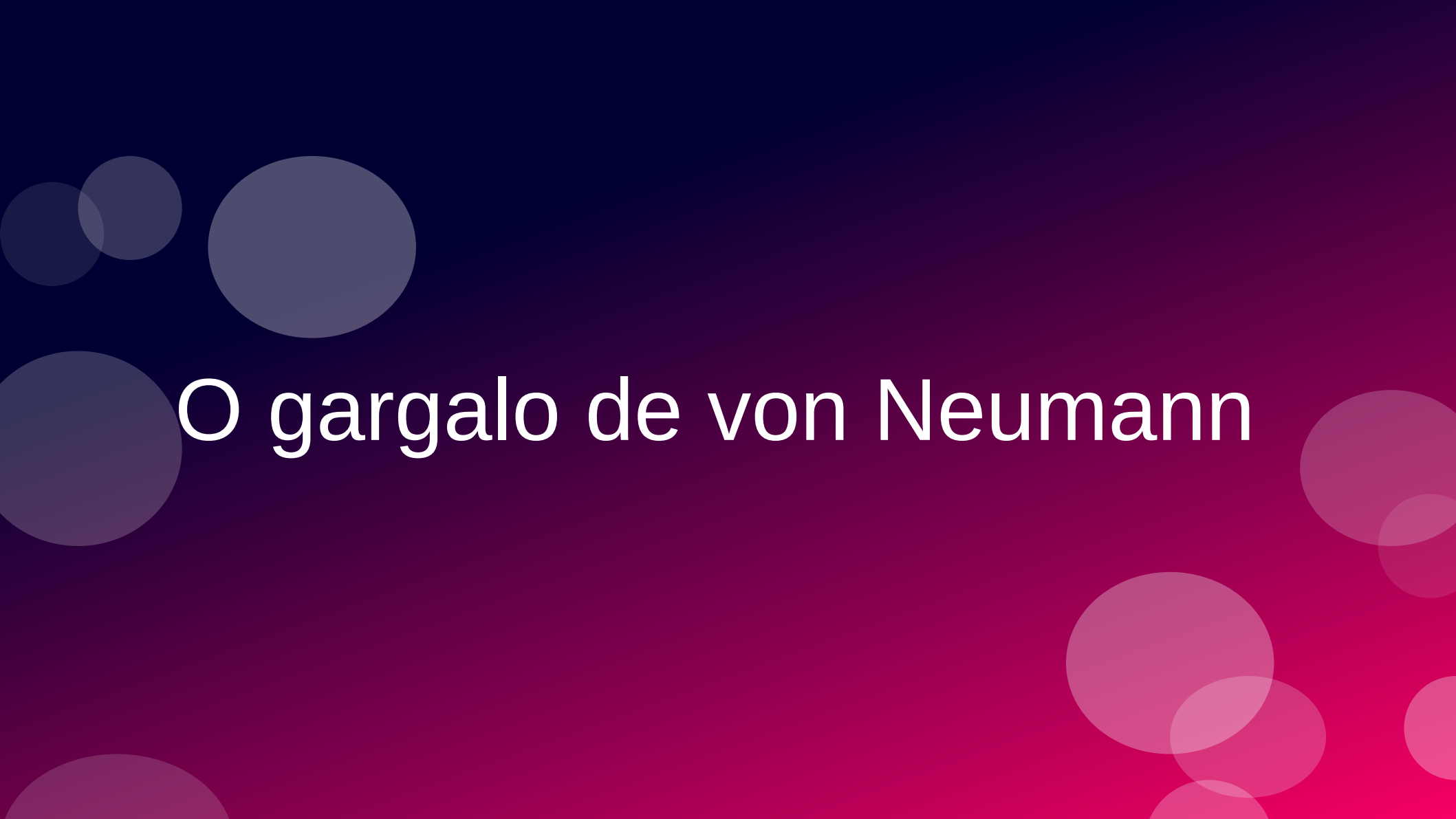
```
// unigo-set
seen := make(set.Set[string])
for lines.Scan() {
    line := lines.Text()
    if seen.Insert(line) {
        fmt.Fprintln(buf, line)
    }
}
```



As três
variantes

Quadratic vs. linear growth





O gargalo de von Neumann



Can Programming Be Liberated from the von Neumann Style? A Functional Style and Its Algebra of Programs

John Backus
IBM Research Laboratory, San Jose



General permission to make fair use in teaching or research of all or part of this material is granted to individual readers and to nonprofit libraries acting for them provided that ACM's copyright notice is given and that reference is made to the publication, to its date of issue, and to the fact that reprinting privileges were granted by permission of the Association for Computing Machinery. To otherwise reprint a figure, table, other substantial excerpt, or the entire work requires specific permission as does republication, or systematic or multiple reproduction.

Author's address: 91 Saint Germain Ave., San Francisco, CA 94114.

© 1978 ACM 0001-0782/78/0800-0613 \$00.75

613

Conventional programming languages are growing ever more enormous, but not stronger. Inherent defects at the most basic level cause them to be both fat and weak: their primitive word-at-a-time style of programming inherited from their common ancestor—the von Neumann computer, their close coupling of semantics to state transitions, their division of programming into a world of expressions and a world of statements, their inability to effectively use powerful combining forms for building new programs from existing ones, and their lack of useful mathematical properties for reasoning about programs.

An alternative functional style of programming is founded on the use of combining forms for creating programs. Functional programs deal with structured data, are often nonrepetitive and nonrecursive, are hierarchically constructed, do not name their arguments, and do not require the complex machinery of procedure declarations to become generally applicable. Combining forms can use high level programs to build still higher level ones in a style not possible in conventional languages.

Communications
of
the ACM

August 1978
Volume 21
Number 8

Ecos do passado

- Em 2018 apresentei “Prática de Conjuntos” na GopherCon Brasil:
 - "Porquê e como implementar um tipo Set em Go"
- Esta é uma atualização profunda, abordando os tipos Set no pacote de coleções do Go 1.28

vintage
2018

ThoughtWorks®

construção e uso

PRÁTICA DE CONJUNTOS

Porquê e como implementar um tipo Set na linguagem Go.



Luciano Ramalho
@ramalhoorg

CASO DE USO #1

EXIBIR PRODUTO SE
TODAS AS PALAVRAS
DA CONSULTA APARE-
CEREM NA DESCRIÇÃO

SOLUÇÃO SEM CONJUNTOS #1

vintage
2018

Já escrevi código assim...

```
func ContainsAll(slice, subslice []string) bool {  
    for _, needle := range subslice {  
        found := false  
        for _, elem := range slice {  
            if needle == elem {  
                found = true  
                break  
            }  
        }  
        if !found {  
            return false  
        }  
    }  
    return true  
}
```

busca em $O(n)$

busca em $O(m)$

execução na ordem de $O(m*n)$

SOLUÇÃO SEM CONJUNTOS #2

vintage
2018

Assim fica mais legível...

```
func Contains(slice []string, needle string) bool {  
    for _, elem := range slice {  
        if needle == elem {  
            return true  
        }  
    }  
    return false  
}
```

busca em $O(n)$

```
func ContainsAll(slice, subslice []string) bool {  
    for _, needle := range subslice {  
        if !Contains(slice, needle) {  
            return false  
        }  
    }  
    return true  
}
```

execução na ordem de $O(m*n)$

SOLUÇÃO SEM CONJUNTOS #2

vintage
2023

Assim fica mais legível...

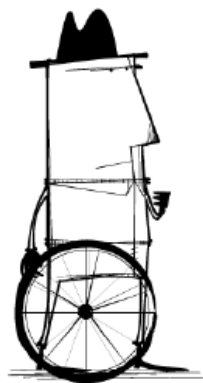
Desde o Go 1.21:

`Contains[S ~[]E, E comparable](s S, v E) bool`

```
func ContainsAll(slice, subslice []string) bool {  
    for _, needle := range subslice {  
        if !Contains(slice, needle) {  
            return false  
        }  
    }  
    return true  
}
```

execução na ordem de $O(m*n)$

Que tal
se...



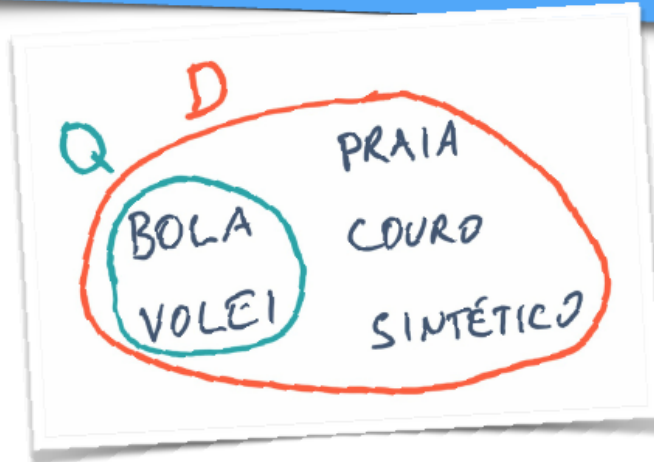
Depois! Agora estou
muito ocupado!



CASO DE USO #1

www.workcompass.com/

EXIBIR PRODUTO SE
TODAS AS PALAVRAS
DA CONSULTA APARE-
CEREM NA DESCRIÇÃO



$$Q \subseteq D$$

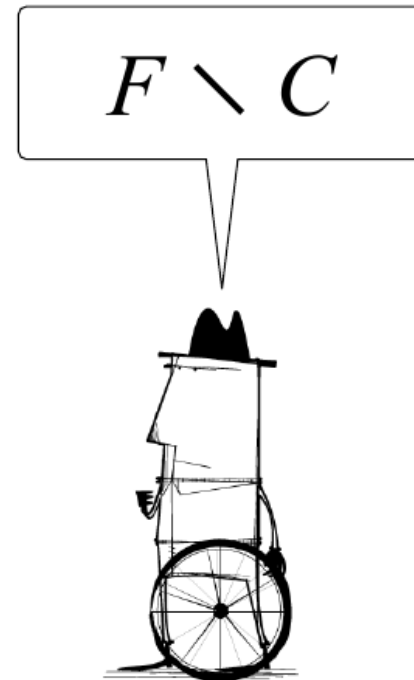
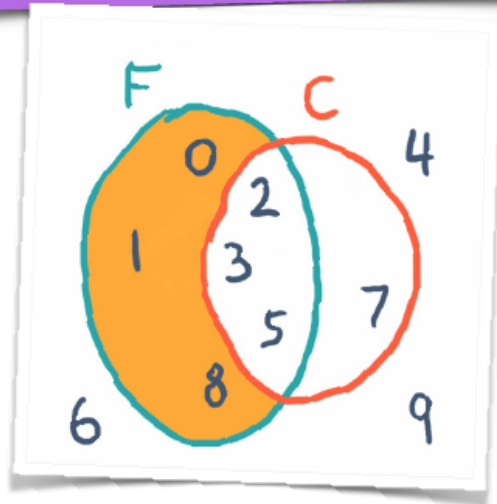


CASO DE USO #2

Marcar todos os produtos favoritados, exceto os que já estiverem no carrinho de compras.

CASO DE USO #2

Marcar todos os produtos favoritados, exceto os que já estiverem no carrinho de compras.





Conjuntos e Lógica

Ainda não descobriram um ramo da
matemática que não possa ser formalizado
pela teoria dos conjuntos.

Thomas Forster
Logic Induction and Sets
p. 167

CONJUNÇÃO LÓGICA: INTERSECÇÃO

x pertence à intersecção de A e B.

é o mesmo que:

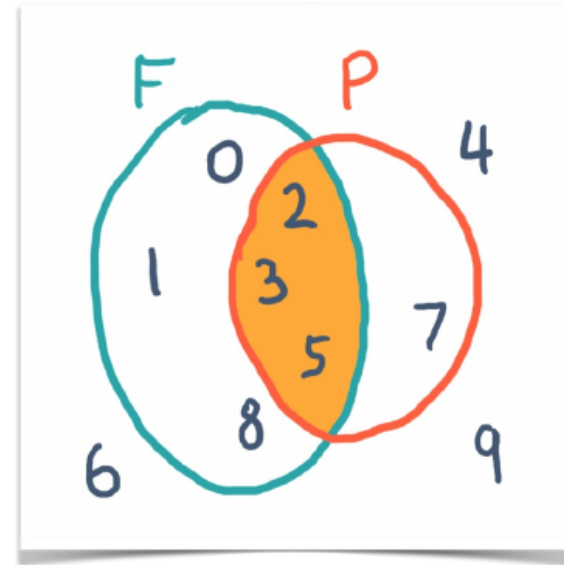
x pertence ao conjunto A e

x também pertence ao conjunto B.

Em notação matemática:

$$x \in (A \cap B) \iff (x \in A) \wedge (x \in B)$$

Em computação: **AND**



DISJUNÇÃO LÓGICA: UNIÃO

x pertence à união de A e B.

é o mesmo que:

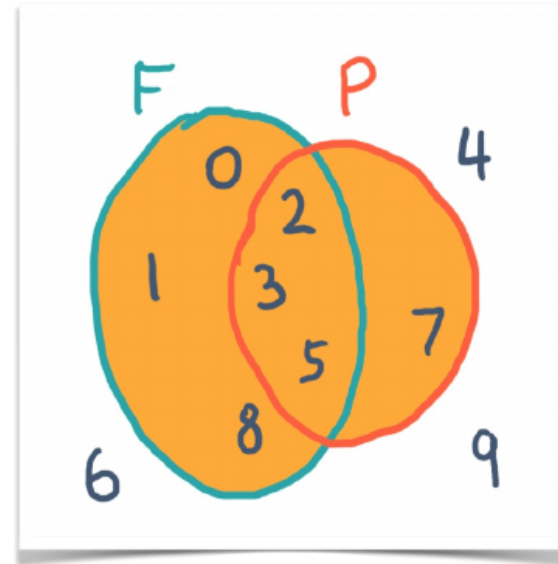
x pertence ao conjunto A e/ou

x pertence ao conjunto B.

Em notação matemática:

$$x \in (A \cup B) \iff (x \in A) \vee (x \in B)$$

Em computação: **OR**



DIFERENÇA

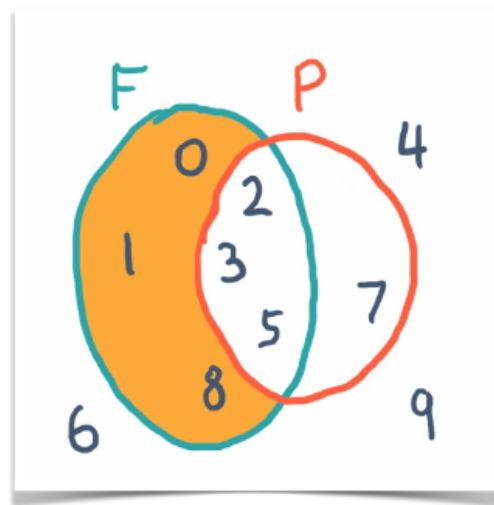
*x pertence ao conjunto A mas
não pertence ao conjunto B.*

é o mesmo que:

elementos de A menos os de B

Em notação matemática:

$$x \in (A \setminus B) \iff (x \in A) \wedge (x \notin B)$$



DIFERENÇA SIMÉTRICA

*x pertence ao conjunto A ou
x pertence ao conjunto B
mas não pertence aos dois*

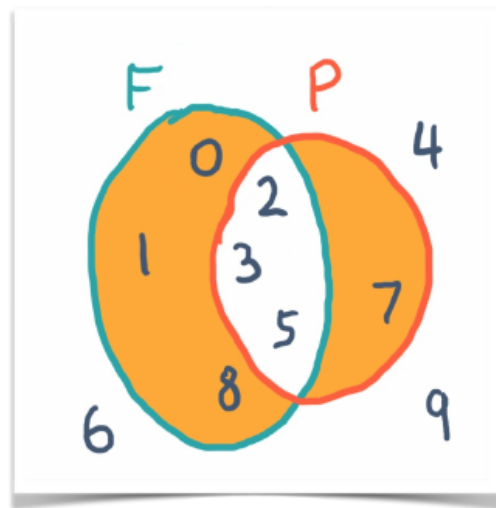
é o mesmo que:

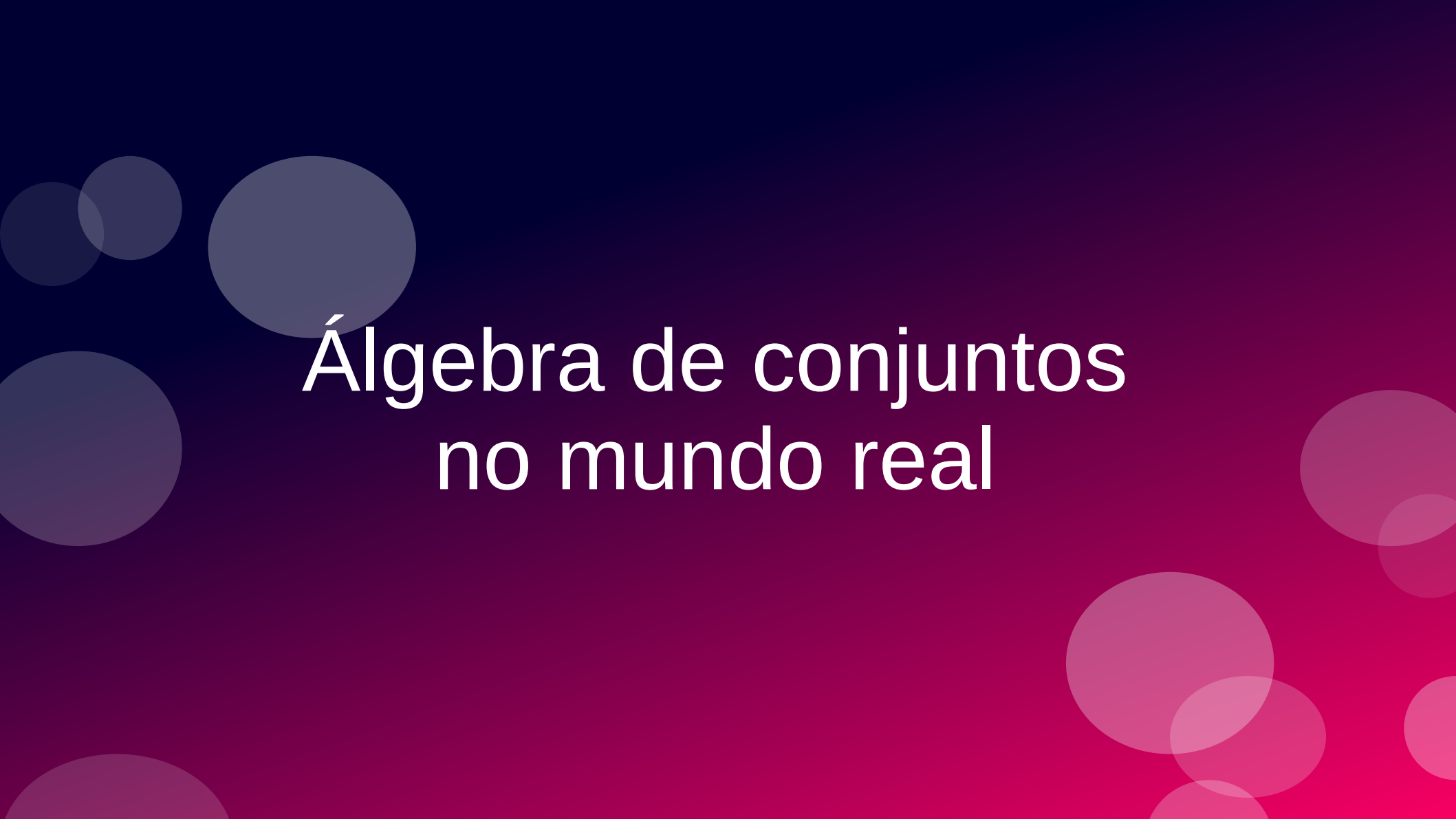
*x pertence à união de A e B menos a
intersecção de A e B.*

Em notação matemática:

$$x \in (A \Delta B) \iff (x \in A) \vee (x \in B)$$

Em computação: **XOR**





Álgebra de conjuntos no mundo real

Operações da álgebra de conjuntos

- Essencial: $e \in S$
- Muito úteis na prática:
 - $S \cap Z$ (interseção)
 - $S \cup Z$ (união)
 - $S \setminus Z$ (diferença)
 - $S \subseteq Z$ (subconjunto)
 - Popular: ContainsAll (todos os elementos de S pertencem a Z)

Implementação de conjuntos

- Invariante: elementos devem ser únicos
 - Elementos precisam ser comparáveis e *hashable*
- $e \in S$ (teste de pertencimento):
 - Espera-se que seja $O(1)$ em uma implementação razoável
- O teste de pertencimento em tempo constante viabiliza ganhos de desempenho importantes ao implementar as demais operações

CONJUNTOS EM VÁRIAS BIBLIOTECAS-PADRÃO

Algumas linguagens/plataformas que implementam conjuntos em sua biblioteca-padrão.

Java Interface **Set** com < 10 métodos; 8 implementações

JavaScript (ES6) **Set** com < 10 métodos

.Net (C# etc.) Interface **ISet** com > 10 métodos; 2 implementações

Python **set** e **frozenset** com > 10 métodos e operadores

Ruby **Set** com > 10 métodos e operadores

vintage
2018

APIs
limitadas

APIs
abrangentes

CONJUNTOS EM VÁRIAS BIBLIOTECAS-PADRÃO

Algumas linguagens/plataformas que implementam conjuntos em sua biblioteca-padrão.

Java Interface **Set** com < 10 métodos; 8 implementações

JavaScript (ES6) **Set** com < 10 métodos

.Net (C# etc.) Interface **ISet** com > 10 métodos; 2 implementações

Python **set** e **frozenset** com > 10 métodos e operadores

Ruby **Set** com > 10 métodos e operadores

vintage
2018

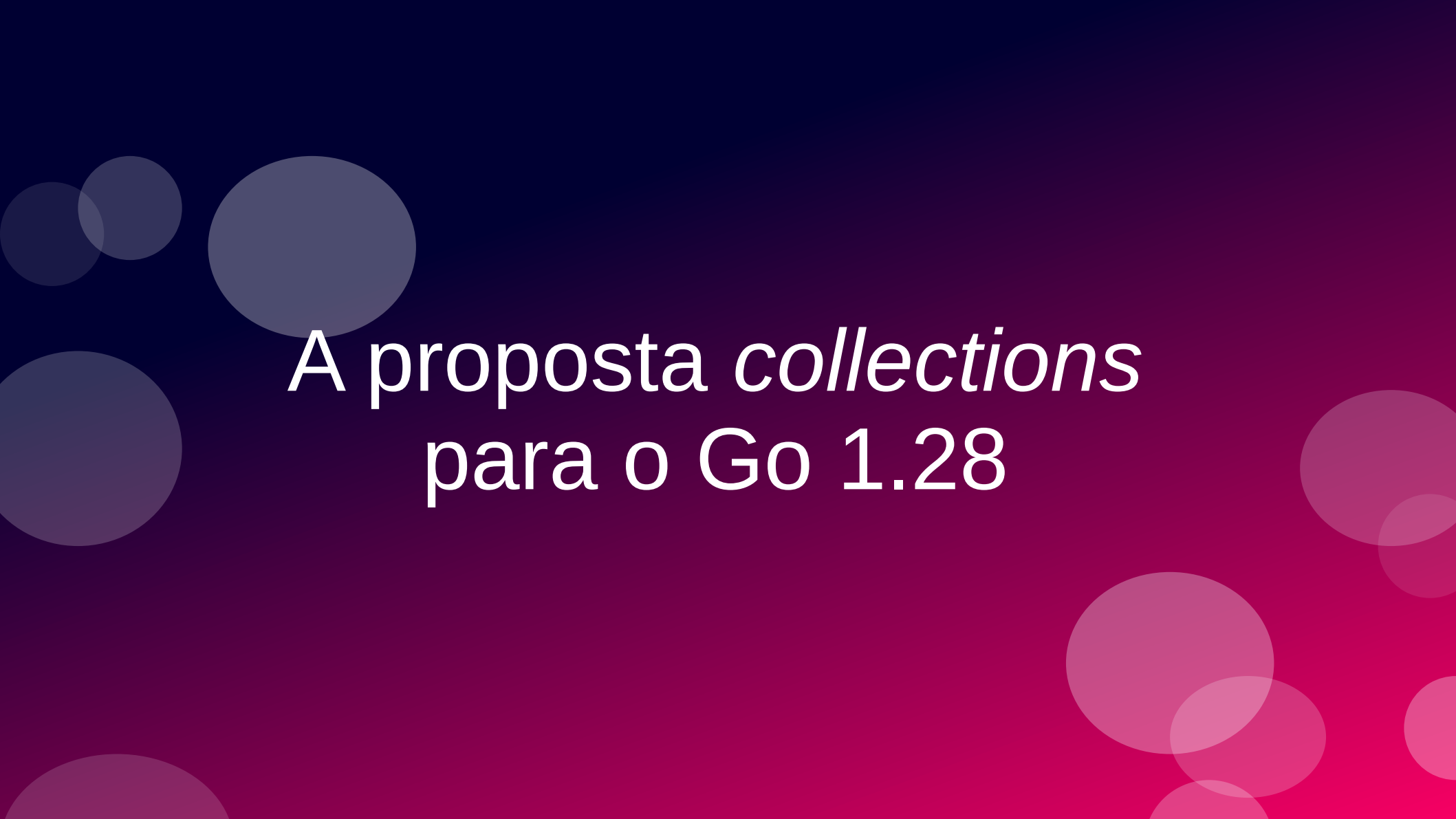
APIs limitadas

- A maioria dos métodos opera com um elemento de cada vez
 - ex.: incluir/remover elemento, iteração
- Poucos ou nenhum método processa conjuntos inteiros (interseção, união etc.)
- Exemplo do *Gargalo de von Neumann*

Evolução das APIs de sets

- Linguagens incorporando mais operações da álgebra de conjuntos
- Exemplo: ECMAScript 2025

```
Set.intersection, Set.union,  
Set.difference,  
Set.symmetricDifference,  
Set.isSubsetOf, Set.isSupersetOf,  
Set.isDisjointFrom
```



A proposta *collections* para o Go 1.28

O grupo de trabalho Go Collections foi formado no final de 2025 com o objetivo de trazer estruturas de dados de coleção comuns para a biblioteca padrão, norteado pelos princípios familiares de pragmatismo e simplicidade do Go.

...

Este trabalho busca adicionar vários dos tipos de dados mais importantes à biblioteca padrão e estabelecer convenções para suas APIs e para as APIs de futuras adições.

Alan Donovan
Go issue #80590

Go Collections Working Group

- Jonathan Amsterdam 🖐️
- Alan Donovan
- Robert Griesemer
- Daniel Martí
- Roger Peppe
- Keith Randall
- Ian Lance Taylor



proposal: container/...: generic collection types

- Umbrella issue 🌂
<https://github.com/golang/go/issues/80590>
- 6 novos tipos concretos de coleções
- 1 nova interface Hasher (já no Go 1.27)
- 3 novas interfaces abstratas para coleções em geral, sets, e maps



Os 7 componentes propostos

1) `hash/maphash.Hasher`

interface padrão que suporta funções de hash customizadas e relações de equivalência para tipos de dados arbitrários (definida no Go 1.27)

2) `container/hash.Map[K, V]`

um map baseado em hash que usa funções de hash customizadas.

3) `container/hash.Set[T]`

a um conjunto baseado em Set, na mesma linha do `hash.Map`

4) `container/heap/v2.Heap`

uma API genérica de heap binário para substituir o heap existente na biblioteca padrão (que é difícil de usar).

Os 7 componentes propostos

5) `container/set.Set[T]`

representado de forma transparente como `map[T]struct{}`, suportando operações de conjunto usuais, como União e Interseção. "Esperamos que se torne o conjunto padrão na maioria das novas APIs em Go."

6) `container/mapset`

funções auxiliares (união, interseção, etc.) para manipular conjuntos legados como conjuntos em código existente cuja API não pode ser alterada. `set.Set` encapsula essa API.

7) `container/ordered.Map[K, V]`

um mapa que preserva a ordem de inserção das chaves.

Source tree (WIP)

Open CIs
(unmerged)
identified
by #issue

go/src/	
└─ hash/	
└─ maphash/	
└─ hasher.go	= Hasher interface, in Go 1.27 (#70471)
└─ maps/	
└─ maps.go	★ added Identical (#78456), used by mapset
└─ container/	
└─ container_test.go	★+ abstract interfaces (unexported)
└─ set/	
└─ set.go	★+ the new canonical set (#69230)
└─ mapset/	
└─ mapset.go	★+ helpers for map-based sets (#77052)
└─ hash/	
└─ map.go	★+ hash.Map, uses maphash.Hasher (#69559)
└─ set.go	★+ hash.Set, same approach (#80584)
└─ heap/	
└─ heap.go	= existing, pre-generics
└─ v2/	
└─ heap.go	? generic, more ergonomic Heap (#77397)
└─ list/	
└─ list.go	= existing
└─ ordered/	
└─ ordered.go	? tree-based ordered.Map/Set (#60630)
└─ ring/	
└─ ring.go	= existing

go/src/

- |— hash/
 - |— |— maphash/
 - |— |— hasher.go
- |— maps/
 - |— |— maps.go
- |— container/
 - |— container/
 - |— |— container_test.go
- |— |— set/
 - |— |— |— set.go
- |— |— mapset/
 - |— |— |— mapset.go
- |— |— hash/
 - |— |— |— map.go
 - |— |— |— set.go

Interfaces abstratas

Não exportadas,
mas apresentadas
como modelos para
novos tipos de
coleções

Comentário em container/container_test.go

```
// The following interfaces define the abstract data types for  
// collections in Go. They are expressed using F-bounded polymorphic  
// interfaces to achieve covariant parameter/result specialization,  
// and may be used as constraint types in generic functions.  
//  
// These interfaces are not yet published, but may be included in a  
// future Go release once we have experience of whether these methods  
// are necessary and sufficient.
```

Comentário em container/container_test.go

```
// As interfaces a seguir definem os tipos abstratos de dados para
// coleções em Go. Elas são expressas utilizando interfaces polimórficas
// com F-bound (F-bounded polymorphism) para permitir a especialização
// covariante de parâmetros e resultados, e podem ser usadas como
// restrições de tipo em funções genéricas.
//
// Essas interfaces ainda não foram publicadas, mas podem ser incluídas
// em uma versão futura do Go, assim que tivermos experiência suficiente
// para determinar se esses métodos são necessários e suficientes.
```

A interface

AbstractSet

```
1 // _AbstractSet models a set S of elements E,  
2 // such as *hash.Set, or set.Set.  
3 type _AbstractSet[E any, S _AbstractSet[E, S]] interface {  
4     _AbstractCollection[E, S]  
5  
6     Insert(E) bool  
7     InsertAll(iter.Seq[E]) bool  
8     Equal(S) bool  
9     All() iter.Seq[E]  
10    Delete(E) bool  
11    DeleteAll(iter.Seq[E]) bool  
12    DeleteFunc(func(E) bool) bool  
13    Intersection(S) S  
14    IntersectionWith(S)  
15    Intersects(S) bool  
16    Union(S) S  
17    UnionWith(S)  
18    Difference(S) S  
19    DifferenceWith(S)  
20    SymmetricDifference(S) S  
21    SymmetricDifferenceWith(S)  
22 }
```



```
1 // _AbstractSet models a set S of elements E,  
2 // such as *hash.Set, or set.Set.  
3 type _AbstractSet[E any, S _AbstractSet[E, S]] interface {  
4     _AbstractCollection[E, S]  
5  
6     Insert(E) bool  
7     InsertAll(iter.Seq[E]) bool  
8     Equal(S) bool  
9     All() iter.Seq[E]  
10    Delete(E) bool  
11    DeleteAll(iter.Seq[E]) bool  
12    DeleteFunc(func(E) bool) bool  
13    Intersection(S) S  
14    IntersectionWith(S)  
15    Intersects(S) bool  
16    Union(S) S  
17    UnionWith(S)  
18    Difference(S) S  
19    DifferenceWith(S)  
20    SymmetricDifference(S) S  
21    SymmetricDifferenceWith(S)  
22 }
```

devolvem
bool para
indicar
mudança

```
1 // _AbstractSet models a set S of elements E,  
2 // such as *hash.Set, or set.Set.  
3 type _AbstractSet[E any, S _AbstractSet[E, S]] interface {  
4     _AbstractCollection[E, S]  
5  
6     Insert(E) bool  
7     InsertAll(iter.Seq[E]) bool  
8     Equal(S) bool  
9     All() iter.Seq[E]  
10    Delete(E) bool  
11    DeleteAll(iter.Seq[E]) bool  
12    DeleteFunc(func(E) bool) bool  
13    Intersection(S) S   
14    IntersectionWith(S)  
15    Intersects(S) bool  
16    Union(S) S   
17    UnionWith(S)  
18    Difference(S) S   
19    DifferenceWith(S)  
20    SymmetricDifference(S) S   
21    SymmetricDifferenceWith(S)  
22 }
```

operações
da álgebra
de conjuntos
devolvem
um novo Set

```
1 // _AbstractSet models a set S of elements E,  
2 // such as *hash.Set, or set.Set.  
3 type _AbstractSet[E any, S _AbstractSet[E, S]] interface {  
4     _AbstractCollection[E, S]  
5  
6     Insert(E) bool  
7     InsertAll(iter.Seq[E]) bool  
8     Equal(S) bool  
9     All() iter.Seq[E]  
10    Delete(E) bool  
11    DeleteAll(iter.Seq[E]) bool  
12    DeleteFunc(func(E) bool) bool  
13    Intersection(S) S  
14    IntersectionWith(S)   
15    Intersects(S) bool  
16    Union(S) S  
17    UnionWith(S)  
18    Difference(S) S  
19    DifferenceWith(S)   
20    SymmetricDifference(S) S  
21    SymmetricDifferenceWith(S)   
22 }
```

modificam o
próprio set
(in-place)

AbstractSet: declaração auto-referente!

```
// _AbstractSet models a set S of elements E,  
// such as *hash.Set, or set.Set.
```

```
type _AbstractSet[E any, S _AbstractSet[E, S]] interface {
```

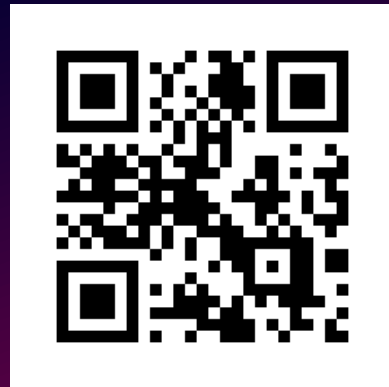


A classe Enum é, na verdade, uma classe genérica definida como `Enum<T> extends Enum<T>>`. Essa definição circular é, provavelmente, a definição de tipo genérico mais intrigante que você encontrará. Especialistas em teoria de tipos nos garantem que isso é perfeitamente válido e significativo — e que simplesmente não devemos pensar muito a respeito —, pelo que somos gratos.



*Ken Arnold, James Gosling, David Holmes
The Java Programming Language, 4th Edition
Citado em Generics Considered Harmful
por Ken Arnold (<https://tgo.li/25>)*

Uma boa explicação informal



- *Polimorfismo F-Bounded:*
Builders com Segurança de Tipo em Java,
por Pradeep Samuel (publicado em 9 de março de 2026)
 - Como uma restrição de tipo autorreferencial resolve um problema real de herança, por que o próprio Enum do Java utiliza esse mesmo truque e o que tudo isso significa na prática.

<https://www.fbounded.com/blog/f-bounded-polymorphism/>

AbstractSet é F-bounded

```
// _AbstractSet models a set S of elements E,  
// such as *hash.Set, or set.Set.
```

```
type _AbstractSet[E any, S _AbstractSet[E, S]] interface {
```

- Esta é a notação em Go para uma interface polimórfica F-bounded
- Fornece a variável de tipo S para as assinaturas de operações genéricas da álgebra de conjuntos
- Limita o tipo concreto de S a um subtipo de _AbstractSet

```
1 // _AbstractSet models a set S of elements E,  
2 // such as *hash.Set, or set.Set.  
3 type _AbstractSet[E any, S _AbstractSet[E, S]] interface {  
4     _AbstractCollection[E, S]  
5  
6     Insert(E) bool  
7     InsertAll(iter.Seq[E]) bool  
8     Equal(S) bool  
9     All() iter.Seq[E]  
10    Delete(E) bool  
11    DeleteAll(iter.Seq[E]) bool  
12    DeleteFunc(func(E) bool) bool  
13    Intersection(S) S  
14    IntersectionWith(S)  
15    Intersects(S) bool  
16    Union(S) S  
17    UnionWith(S)  
18    Difference(S) S  
19    DifferenceWith(S)  
20    SymmetricDifference(S) S  
21    SymmetricDifferenceWith(S)  
22 }
```

usos da
variável S
como tipo
de retorno

_AbstractMap também é F-bounded

```
1 // _AbstractMap models a mapping M from keys K to values V,  
2 // such as *hash.Map or *ordered.Map.  
3 type _AbstractMap[K, V any, M _AbstractMap[K, V, M]] interface {  
4     _AbstractCollection[K, M]  
5  
6     Set(K, V) (V, bool)  
7     SetAll(iter.Seq2[K, V]) bool  
8     Get(K) (V, bool)  
9     At(K) V  
10    All() iter.Seq2[K, V]  
11    Keys() iter.Seq[K]  
12    Values() iter.Seq[V]  
13    Delete(K) (V, bool)  
14    DeleteAll(iter.Seq[K]) bool  
15    DeleteFunc(func(K, V) bool) bool  
16 }
```

AbstractCollection, a interface-base

```
1 // _AbstractCollection models a collection C of elements E,  
2 // such as *hash.Map, *hash.Set, *ordered.Map, or set.Set.  
3 type _AbstractCollection[E any, C _AbstractCollection[E, C]] interface {  
4     Clear()  
5     Clone() C  
6     Contains(E) bool  
7     ContainsAll(iter.Seq[E]) bool  
8     Len() int  
9     String() string  
10 }
```

Tanto `_AbstractSet` como `_AbstractMap`
embutem `_AbstractCollection`

Abaixo das interfaces em container_test.go

```
// These types define the fundamental operations that need to be
// implemented by all set and map types. Their naming, signature, and
// semantic conventions should be followed wherever possible when
// defining new collection types.
// ...
//
// Map.Set should replace an existing entry with an equivalent key.
// This follows the built-in map: https://go.dev/play/p/pkH8kkFTuEg.
//
// The "plural" functions {Contains,Delete,Insert,Set}All are
// sufficiently important that they belong as methods; they
// also compute a convenient bool result.
```

Abaixo das interfaces em `container_test.go`

```
// Estes tipos definem as operações fundamentais que precisam ser
// implementadas por todos os tipos de conjunto (set) e mapa (map).
// Suas convenções de nomenclatura, assinatura e semântica devem ser
// seguidas sempre que possível ao definir novos tipos de coleção.
// ...
//
// Map.Set deve substituir uma entrada existente que tenha uma chave
// equivalente. Isso segue o comportamento do mapa nativo:
// https://go.dev/play/p/pkH8kkFTuEg.
//
// As funções "plurais" {Contains,Delete,Insert,Set}All são
// importantes o suficiente para serem implementadas como métodos;
// elas também retornam um resultado booleano útil.
```

Testes estáticos container_test.go

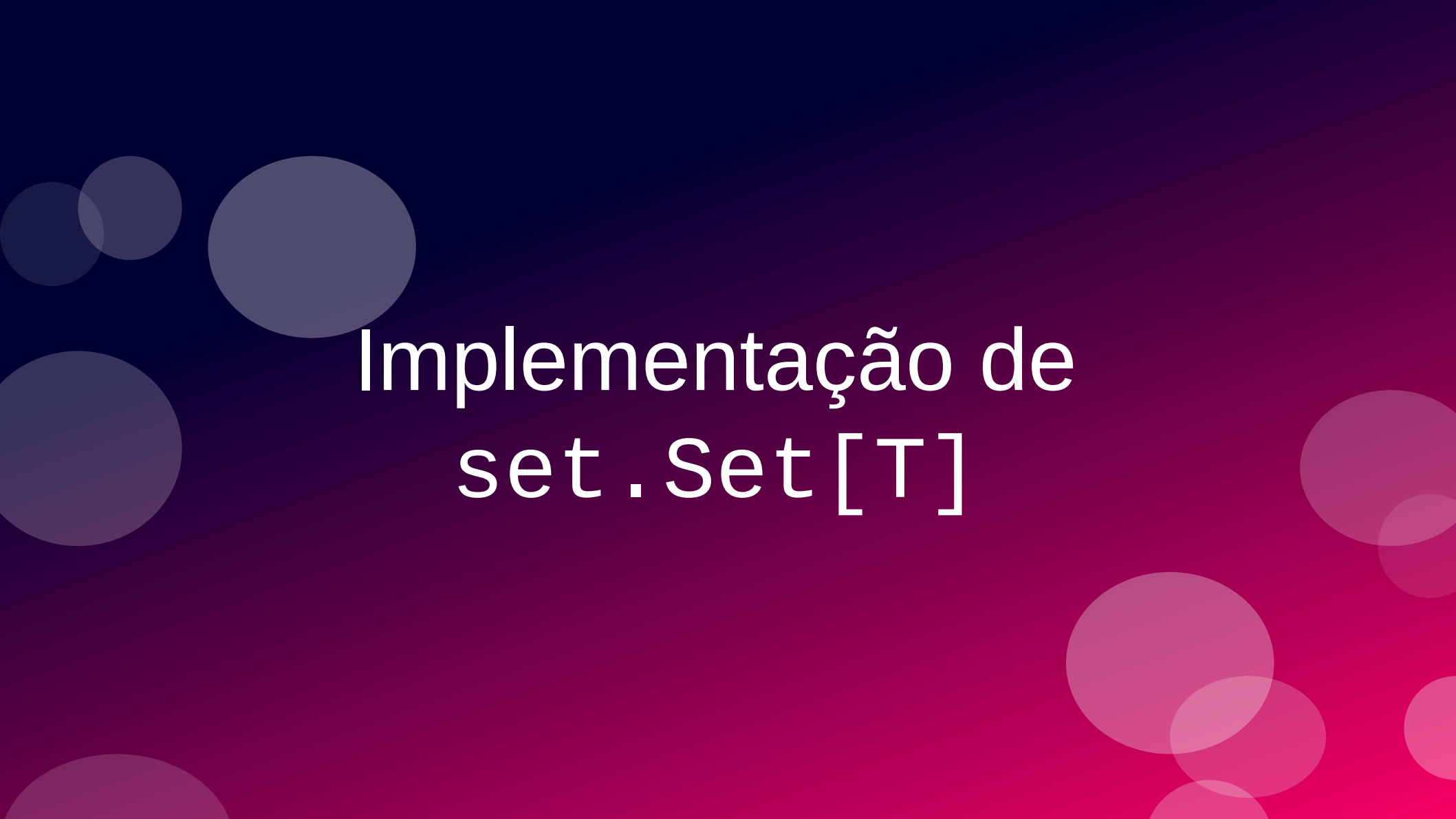
```
1 // -- conformance --
2
3 // This is a static compilation test of various symmetries,
4 // expressed using F-bounded polymorphic interfaces to
5 // achieve covariant parameter/result specialization.
6
7 var _ _AbstractSet[int, set.Set[int]] = make(set.Set[int])
8
9 var _ _AbstractSet[int, *hash.Set[int]] = new(hash.Set[int])
```

Este teste verifica que o compilador aceita atribuir `set.Set` e `hash.Set` a uma variável `_` do tipo `_AbstractSet`

Função genérica em container_test.go

```
1 // Take removes and returns an arbitrary element from a set.
2 // It returns zero if the set was empty.
3 func Take[S _AbstractSet[E, S], E any](set S) (e E, found bool) {
4     for e = range set.All() {
5         found = true
6         set.Delete(e) // may fail for NaN
7         break
8     }
9     return
10 }
```

Take devolve um elemento do tipo E
(declarado em `_AbstractSet[E, S]`) e um bool



Implementação de `set.Set[T]`

```
go/src/  
├── hash/  
│   └── maphash/  
│       └── hasher.go  
├── maps/  
│   └── maps.go  
├── container/  
├── container/  
│   ├── container_test.go  
│   ├── set/  
│   │   └── set.go  
│   ├── mapset/  
│   │   └── mapset.go  
│   └── hash/  
│       ├── map.go  
│       └── set.go
```

set.Set[T]

**Veja o código-fonte da
proposta em /vendored
no repositório GitHub
ramalho/sets-in-go**



set.Set delega tudo para funções de mapset

```
1 package set
2
3 import (
4     "mapset" // proposed mapset in Go 1.28
5     "iter"
6     "maps"
7 )
8
9 // A Set[E] is a set of elements of type E.
10 type Set[E comparable] map[E]struct{}
11
12 // Collect creates a new set containing the elements of the sequence.
13 func Collect[E comparable](seq iter.Seq[E]) Set[E] {
14     return Set[E](mapset.Collect(seq))
15 }
```

Métodos de uma linha que apenas invocam funções de mapset

```
go/src/  
├── hash/  
│   └── maphash/  
│       └── hasher.go  
├── maps/  
│   └── maps.go  
├── container/  
├── container/  
│   ├── container_test.go  
│   ├── set/  
│   │   └── set.go  
│   ├── mapset/  
│   │   └── mapset.go  
│   └── hash/  
│       ├── map.go  
│       └── set.go
```

mapset.go
funções para usar
um mapa legado
como um conjunto

mapset.go não define nenhum tipo com nome

```
1 package mapset
2
3 import (
4     "iter"
5 )
6
7 // Collect returns a new set containing the elements of the sequence.
8 func Collect[K comparable](seq iter.Seq[K]) map[K]struct{} {
9     return collect[K, struct{}](seq)
10 }
11
12 /// ...
13
14 func collect[K comparable, V bool | struct{}](seq iter.Seq[K]) map[K]V {
15     x := make(map[K]V)
16     InsertAll(x, seq)
17     return x
18 }
```

set.Collect versus set.Of

```
1 // Collect creates a new set containing the elements of the sequence.
2 func Collect[E comparable](seq iter.Seq[E]) Set[E] {
3     return Set[E](mapset.Collect(seq))
4 }
5
6 // Of creates a new set containing the elements of the sequence.
7 func Of[E comparable](elems ...E) map[E]struct{} {
8     return Set[E](mapset.Of(elems...))
9 }
```

- set.Collect recebe iter.Seq[E]; set.Of recebe ...E
- set.Collect devolve Set[E]; set.Of devolve map[E]struct{}
 - Sinal de WIP! (trabalho em andamento)

Implementation of mapset.Collect

```
1 // Collect returns a new set containing the elements of the sequence.
2 func Collect[K comparable](seq iter.Seq[K]) map[K]struct{} {
3     return collect[K, struct{}](seq)
4 }
5
6 // CollectBool returns a new set containing the elements of the sequence.
7 // The map values are all "true".
8 func CollectBool[K comparable](seq iter.Seq[K]) map[K]bool {
9     return collect[K, bool](seq)
10 }
11
12 func collect[K comparable, V bool | struct{}](seq iter.Seq[K]) map[K]V {
13     x := make(map[K]V)
14     InsertAll(x, seq)
15     return x
16 }
```

Implementação de mapset.Insert{All}

```
1 // Insert adds elem element to the set.
2 // If the set values are boolean, the value 'true' is used.
3 // It reports whether len(x) changed.
4 func Insert[M ~map[K]V, K comparable, V bool | struct{}](x M, elem K) bool {
5     pre := len(x)
6     insert(x, elem)
7     return len(x) != pre
8 }
9
10 // InsertAll adds each element of the addenda sequence to the set.
11 // If the set values are boolean, the value 'true' is used.
12 // It reports whether len(x) changed.
13 func InsertAll[M ~map[K]V, K comparable, V bool | struct{}](x M, addenda iter.Seq[K])
    bool {
14     pre := len(x)
15     for k := range addenda {
16         insert(x, k)
17     }
18     return len(x) != pre
19 }
```

Implementação de mapset.insert

```
1 func insert[M ~map[K]V, K comparable, V bool | struct{}](m M, k K) {
2     // Choose the distinguished "present" value (true or struct{}{}).
3     // This compiles to a load from .rodata.
4     var present V
5     if _, ok := any(present).(bool); ok {
6         present = any(true).(V)
7     }
8
9     // This is the canonical insertion operation.
10    // All maps created by this API use only the
11    // distinguished 'present' value for the result type.
12    m[k] = present
13 }
```

Implementação de `set.Intersection{With}`

```
1 // Intersection returns new map containing the intersection of x and y.
2 func (x Set[E]) Intersection(y Set[E]) Set[E] {
3     return mapset.Intersection(x, y)
4 }
5
6 /// ...
7 // -- in-place binary updates --
8
9 // IntersectionWith updates x to the [Intersection] of x and y.
10 func (x Set[E]) IntersectionWith(y Set[E]) {
11     mapset.IntersectionWith(x, y)
12 }
```

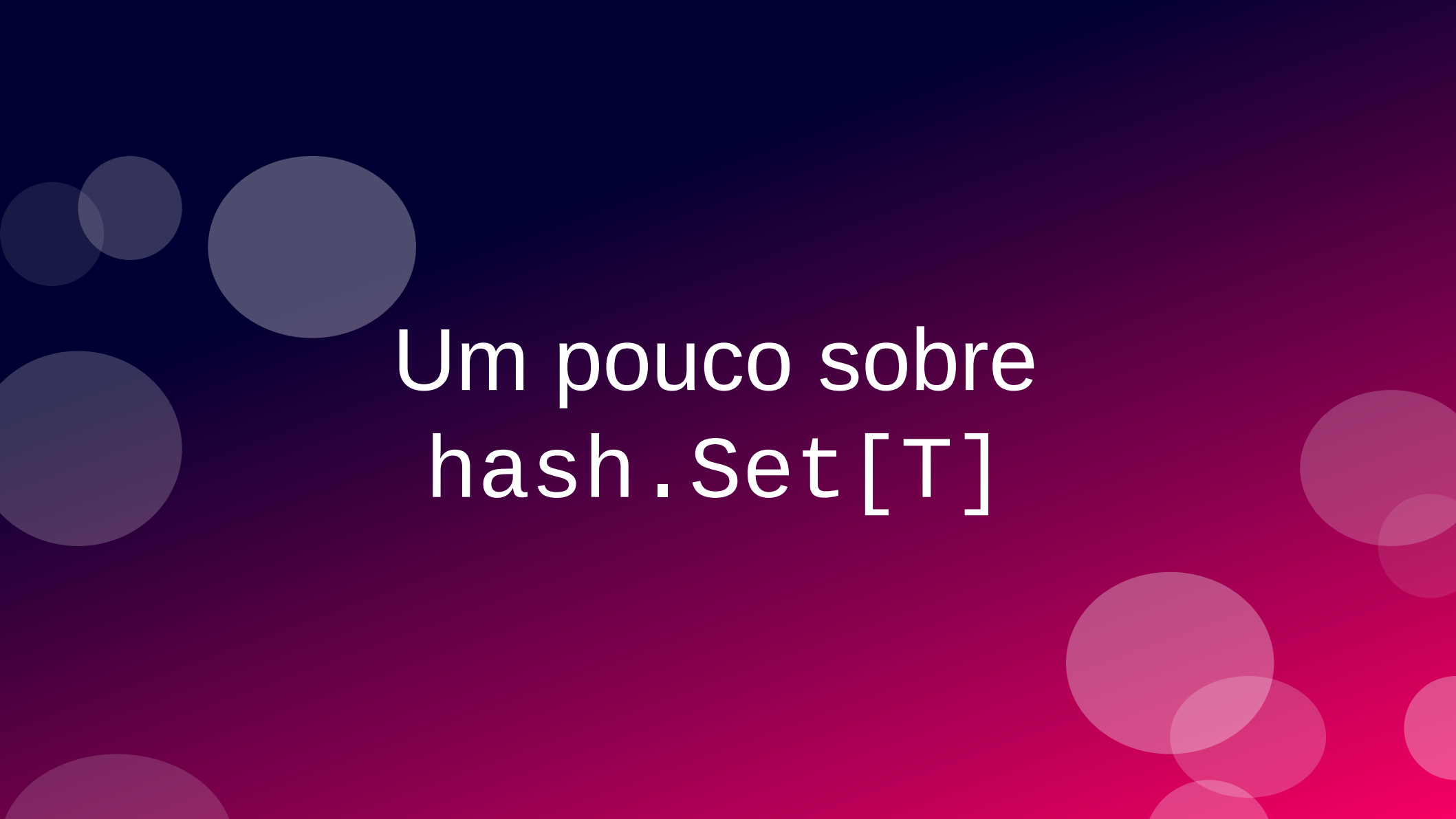

Implementação de mapset.Intersection (1)

```
1 // Intersection returns new map containing the intersection of x and y.
2 func Intersection[MX ~map[K]VX, MY ~map[K]VY,
   K comparable, VX, VY bool | struct{}](x MX, y MY) MX {
3     z := make(MX)
4
5     if maps.Same(x, y) {
6         copy(z, x)
7         return z
8     }
9
10    // Iterate over the smaller of the two maps...
```

continues...

Implementação de mapset.Intersection (2)

```
10 // Iterate over the smaller of the two maps.
11 if len(x) < len(y) {
12     for k := range x {
13         if Contains(y, k) {
14             insert(z, k)
15         }
16     }
17 } else {
18     for k := range y {
19         if Contains(x, k) {
20             insert(z, k)
21         }
22     }
23 }
24 return z
25 }
```



Um pouco sobre `hash.Set[T]`

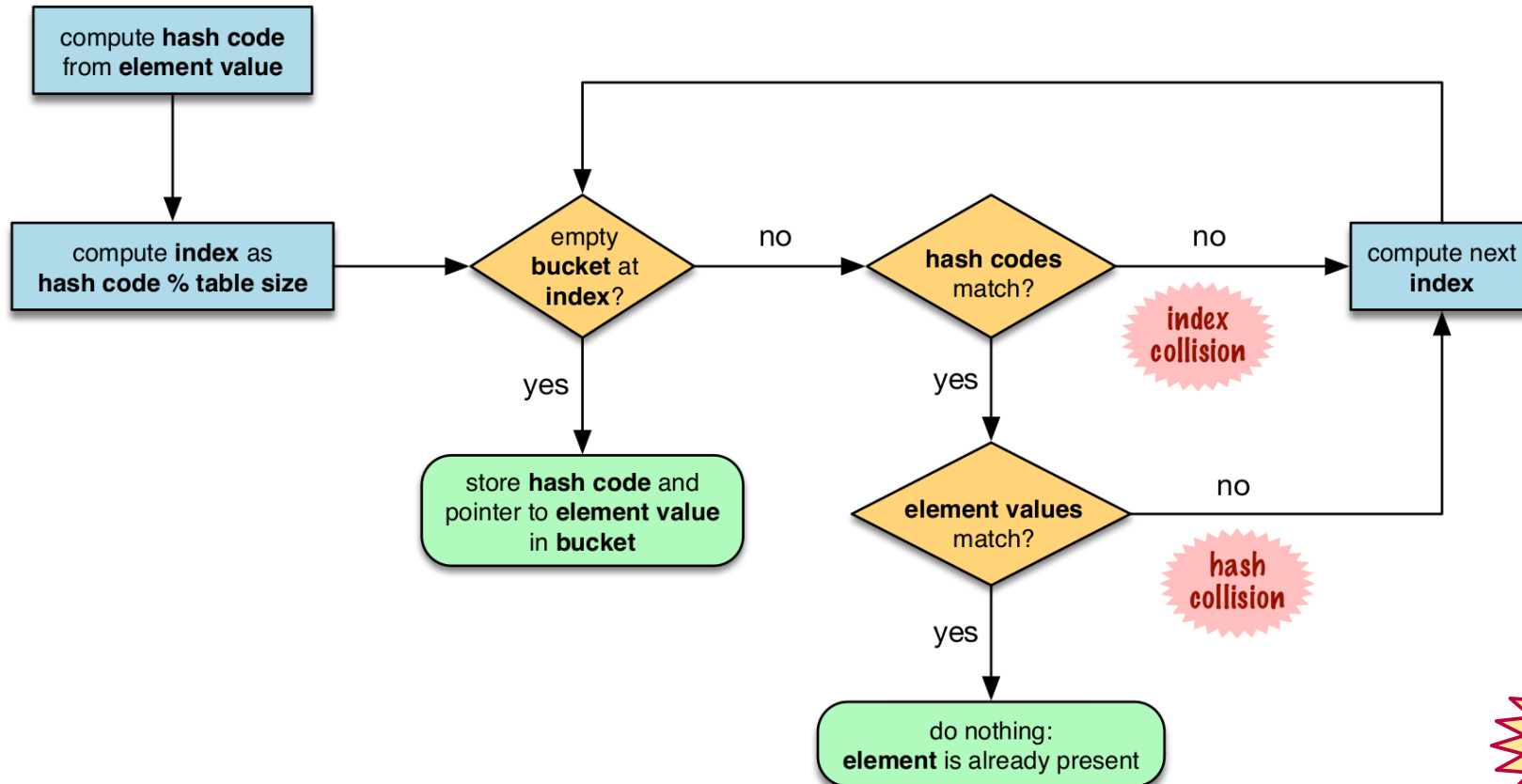
Porquê `hash.Map[K, V]` e `hash.Set[V]`

- Mapas nativos exigem chaves comparable (suportam operador `==`) e uma função de hash embutida no runtime.
- Isso também impõe limitações aos valores em `set.Set[V]`.
- Os tipos `container/hash.Map[K, V]` e `container/hash.Set[V]` do Go 1.28 suportam testes de igualdade e funções de hash personalizados.
- Exemplos:
 - um conjunto de ``big.Int``,
 - um mapa com chaves do tipo `string` que não diferenciam maiúsculas de minúsculas.



Algoritmo de hash table

Algoritmo de hash table (simplificado)



Python
set

Set visível: exibe o estado da hash table



JupyterLab toolbar icons including file, edit, view, and run buttons.

JupyterLab Pyth

VisiSet

A set-like collection backed by a numpy structured array with two columns: `hash` (`int64` or `int32`) and `value` (`object`).

Jupyter renders instances as numpy-style tables showing the internal hash/value pairs.

Demo with strings

```
[1]: from visiset import VisiSet

v = VisiSet()
v
```

Python

A tabela de hash começa com 8 *buckets*

JupyterLab Python 3 (ipykernel)

Demo with strings

```
[1]: from visiset import VisiSet

v = VisiSet()
v
```

```
[1]:
```

%	64-bit hash	pointer to value
	ffffffffffffffff	×
	ffffffffffffffff	×
	ffffffffffffffff	×
	ffffffffffffffff	×
	ffffffffffffffff	×
	ffffffffffffffff	×
	ffffffffffffffff	×
	ffffffffffffffff	×
	ffffffffffffffff	×



Para inserir “Alice”

- Computar o hash da string: 93a4771ea9a10722
- Determinar bucket ($\text{hash} \% 8 = 2$)
- Bucket vazio: colocar hash e valor no bucket

```
JupyterLab Python 3 (ipykernel)
```

```
[2]: v.add('Alice')  
v
```

```
[2]:
```

%	64-bit hash	pointer to value
	ffffffffffffffff	×
	ffffffffffffffff	×
2	93a4771ea9a10722	→ 'Alice'
	ffffffffffffffff	×
	ffffffffffffffff	×
	ffffffffffffffff	×
	ffffffffffffffff	×
	ffffffffffffffff	×



Inserir “Bob”: cai num bucket vazio

```

[3]: v.add('Bob')
v

```

```

[3]: %

```

	64-bit hash	pointer to value
	ffffffffffffffff	×
1	61a9e66141e00411	→ 'Bob'
2	93a4771ea9a10722	→ 'Alice'
	ffffffffffffffff	×
	ffffffffffffffff	×
	ffffffffffffffff	×
	ffffffffffffffff	×
	ffffffffffffffff	×



Inserir “Carla”: outro bucket vazio

```

[4]: v.add('Carla')
v

```

%	64-bit hash	pointer to value
	ffffffffffffffff	×
1	61a9e66141e00411	→ 'Bob'
2	93a4771ea9a10722	→ 'Alice'
3	107047ed7a90c393	→ 'Carla'
	ffffffffffffffff	×
	ffffffffffffffff	×
	ffffffffffffffff	×
	ffffffffffffffff	×

Python

Inserir “Diego”: mais um bucket vazio

Markdown ▾

JupyterLab  Python 3 (ipykernel) 

```
[5]: v.add('Diego')  
v
```

```
[5]: %
```

	64-bit hash	pointer to value
	ffffffffffffffff	×
1	61a9e66141e00411	→ 'Bob'
2	93a4771ea9a10722	→ 'Alice'
3	107047ed7a90c393	→ 'Carla'
	ffffffffffffffff	×
	ffffffffffffffff	×
	ffffffffffffffff	×
7	b8febc5c7e6374af	→ 'Diego'

Python

Inserir “Elise”: bucket ocupado por “Diego”

- Colisão de índice: índice aponta para um hash diferente
 - incrementar índice até encontrar um bucket vazio (com wrap)

```

[6]: v.add('Elise')
v

```

```

[6]: %

```

%	64-bit hash	pointer to value
7	384e354cdf3a183f	→ 'Elise'
1	61a9e66141e00411	→ 'Bob'
2	93a4771ea9a10722	→ 'Alice'
3	107047ed7a90c393	→ 'Carla'
	ffffffffffffffff	×
	ffffffffffffffff	×
	ffffffffffffffff	×
7	b8febc5c7e6374af	→ 'Diego'



Inserir “Fred” dispara a ampliação da tabela

- Buckets vazios ajudam a minimizar o número de colisões
- Ao reinserir os itens na nova tabela, a ordem pode mudar

Python

```
[7]: v.add('Fred')  
v
```

[7]:	%	64-bit hash	pointer to value
	15	b8febc5c7e6374af	→ 'Diego'
	1	61a9e66141e00411	→ 'Bob'
	2	93a4771ea9a10722	→ 'Alice'
	3	107047ed7a90c393	→ 'Carla'
		ffffffffffffffff	×
		ffffffffffffffff	×
		ffffffffffffffff	×
	7	8fc5034bf567fc77	→ 'Fred'
		ffffffffffffffff	×
		ffffffffffffffff	×
		ffffffffffffffff	×
		ffffffffffffffff	×
		ffffffffffffffff	×
		ffffffffffffffff	×
		ffffffffffffffff	×
	15	384e354cdf3a183f	→ 'Elise'

v.union(iterável)

- Cria um novo set com a união dos elementos de v mais os itens do iterável

Python

```
[8]: v.union('Gina Homer Ian Jane'.split())
```

%	64-bit hash	pointer to value
15	384e354cdf3a183f	→ 'Elise'
1	61a9e66141e00411	→ 'Bob'
2	93a4771ea9a10722	→ 'Alice'
3	107047ed7a90c393	→ 'Carla'
	ffffffffffffffff	×
	ffffffffffffffff	×
6	bcef4f45dc253456	→ 'Gina'
7	8fc5034bf567fc77	→ 'Fred'
8	06d85e59cd4796b8	→ 'Jane'
9	15fc23cd5e30d069	→ 'Ian'
	ffffffffffffffff	×
11	3d58c31ef29a89eb	→ 'Homer'
	ffffffffffffffff	×
	ffffffffffffffff	×
	ffffffffffffffff	×
15	b8febc5c7e6374af	→ 'Diego'

Implementação proposta para `hash.Set[T]` no Go 1.28

Tipos Map e Set com suporte a *hashers* customizados

```
go/src/
├── hash/
│   └── maphash/
│       └── hasher.go
├── maps/
│   └── maps.go
├── container/
├── container/
│   ├── container_test.go
│   ├── set/
│   │   └── set.go
│   ├── mapset/
│   │   └── mapset.go
│   └── hash/
│       ├── map.go
│       └── set.go
```

Como customizar Hash e Equal

- Invariante: o hash de objetos iguais tem que ser igual
- Crie uma struct que implementa a interface `maphash.Hasher` introduzida no Go 1.27 em `hash/maphash/maphash.go`
- Passe a struct a um construtor. Propostos no Go 1.28:
 - `hash/map.NewMap`
 - `hash/set.NewSet`

```
func NewSet[E any](hasher maphash.Hasher[E]) *Set[E] {
```

A interface Hasher

```
go/src/  
├── hash/  
│   └── maphash/  
│       └── hasher.go  
├── maps/  
│   └── maps.go  
├── container/  
├── container/  
│   ├── container_test.go  
│   ├── set/  
│   │   └── set.go  
│   ├── mapset/  
│   │   └── mapset.go  
│   └── hash/  
│       ├── map.go  
│       └── set.go
```

A interface `maphash.Hasher` no Go 1.27

```
1 package maphash
2
3 // A Hasher defines the interface between a hash-based container
4 // and its elements. It provides a hash function and an equivalence
5 // relation over values of type T, enabling those values to be
6 // inserted in hash tables and similar data structures.
7 //
8 // ...more than 100 lines of comments...
9
10 type Hasher[T any] interface {
11     Hash(*Hash, T)
12     Equal(x, y T) bool
13 }
```

Exemplo: Hasher *case-insensitive*

```
1 // CaseInsensitive is a Hasher[string] whose
2 // equivalence relation ignores letter case.
3 type CaseInsensitive struct{}
4
5 func (CaseInsensitive) Hash(h *Hash, s string) {
6     h.WriteString(strings.ToLower(s))
7 }
8
9 func (CaseInsensitive) Equal(x, y string) bool {
10     // (We avoid strings.EqualFold as it is not
11     // consistent with ToLower for all values.)
12     return strings.ToLower(x) == strings.ToLower(y)
13 }
```



Finalmentes

Conteúdo obrigatório sobre IA

- Para desenvolvedores que utilizam agentes:
O vocabulário preciso da álgebra de conjuntos é útil para instruir agentes, em qualquer linguagem de programação.
- Para programadores cidadãos:
Aprender álgebra de conjuntos e o modelo relacional pode ser a melhor base para especificar sistemas para agentes
- Para todos:
Exija seu tempo de volta.
Tempo não é dinheiro; é o seu tempo de viver!

Vitasuco de conhecimento

SAM ALTMAN: Vamos fazer um vitasuco
Quem pode contribuir com frutas?

TODOS: Legal, aqui estão minhas frutas!

SAM ALTMAN: O vitasuco está pronto.
Custa \$ 20 o copo!

TODOS: Mas você usou nossas frutas para
fazê-lo!

SAM ALTMAN: Não usei, não!
Cadê suas frutas?
Me mostre!





Não esqueça de Aaron Swartz

- Co-criador do RSS e do Markdown
- Co-fundador do Reddit
- Aaron foi preso em 2011 por baixar em massa artigos científicos do JSTOR
- Sua vida foi destruída pelo Departamento de Justiça dos EUA
- Ele não compartilhou os artigos com ninguém.
- O que iria fazer com eles?
Talvez treinar um modelo?



Uma solução regulatória

- Qualquer operador de LLM que se recuse a divulgar todas as fontes externas utilizadas no treinamento deve disponibilizar o modelo e seus pesos como código aberto.
- Radical demais?
- É apenas justo e bom senso.
- Sem isso, acabou o copyright.



Pontos principais

- A álgebra de conjuntos oferece soluções simples e eficientes para tarefas comuns de processamento de dados.
- Conjuntos e outras coleções na futura versão Go 1.28 servem de base para o design de coleções genéricas.
- Os novos conjuntos e mapas baseados em Hasher são mais flexíveis, podem usar types arbitrárias como elementos ou chaves.
- **O código-fonte de Go apresentado aqui ainda não foi integrado e pode sofrer alterações!**



Slides: <https://speakerdeck.com/ramalho>